Department of Computer Science and Engineering

MavHousing

CSE Senior Design Project-A student housing platform

Khatri, A., Jha, A., Rahman, A., Saxena, A., Tahmid, T., Sami, M.

Advisor: Chris Conly

Code and Materials

Background

University housing operations at UTA are currently managed through multiple disconnected systems covering applications, leasing, maintenance, payments, and communication. This fragmented approach results in inefficient workflows, redundant data entry, delayed approvals, and limited end-to-end visibility across the student housing lifecycle.

As housing demand grows, there is a clear need for a unified, role-based system that streamlines processes and provides a single source of truth for Students, Staff, and Administrators throughout the entire housing experience — from application to move-out.

To address these challenges, we developed MavHousing, a centralized full-stack platform designed to unify and modernize the entire student housing workflow.

Key Requirements

Functional Requirements

- Students can apply for housing, choose preferences, and submit maintenance requests
- System verifies identity using uploaded ID scans
- Staff can review applications, approve leases, and manage maintenance tasks
- Leasing system supports unit/room/bed-level assignments
- Students can view balances, make payments, and receive alerts
- AI assistant answers housing questions through natural language chat
- Admins can generate occupancy and financial reports
- Users can upload and manage documents and media files securely

Non-Functional Requirements

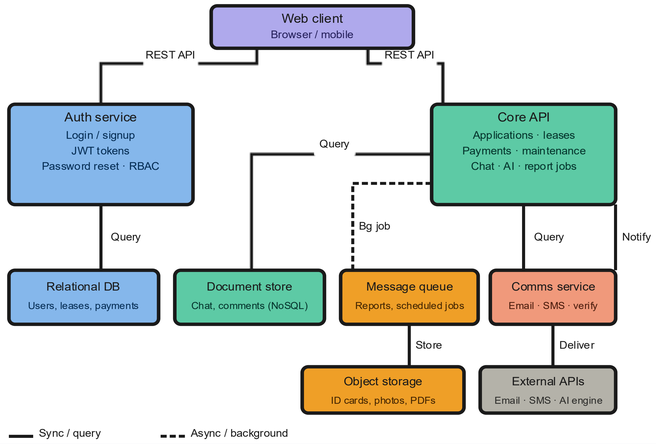
- Role-based access control for all users
- Secure password storage with hashing
- Session authentication with auto-expiry
- Responsive design across devices
- Background jobs must not block the UI
- APIs are documented and testable
- High availability with no single point of failure

Architectural Design

MavHousing is built using a microservice architecture with independent services for authentication, core operations, and communications. It uses a dual-database approach to separate transactional and real-time data, while background workers handle heavy tasks asynchronously to ensure system responsiveness.

Data Strategy

- Relational DB → Stores structured housing records such as users, applications, leases, and payments.
- Document Store → Stores flexible real-time data such as chat messages and comments.
- Message Queue → Handles background jobs like report generation without slowing the system.
- Object Storage → Stores uploaded files and generated reports securely.



Microservices Architecture

Service	Responsibility
auth-server	Centralized auth, session security & RBAC
internal-api	Core hub: housing, leases, maintenance & financials
comms-server	Async transactional email & automated notifications
mcp-agent	AI agent for NL system interactions via MCP & email
web	Next.js portal for students & admin staff

Implementation Details and Test Plan

Implementation Details

MavHousing is engineered as a TypeScript monorepo utilizing NestJS microservices and a Next.js frontend. The platform implements a polyglot data layer, pairing PostgreSQL (managed via Prisma ORM) for strictly relational business logic with MongoDB (via Mongoose) for flexible real-time chat storage. To maintain performance, heavy computational tasks are offloaded to asynchronous background queues using Redis and BullMQ. The system integrates advanced capabilities, including Google Gemini 2.5 and face-api.js for automated OCR identity verification, Cloudflare R2 for secure S3-compatible object storage and resend for transactional emails. Internal data routing is handled via RESTful APIs (documented via Swagger), GraphQL endpoints, and Socket.IO for real-time event streaming.

Test Plan

API Testing: Swagger UI for endpoint validation
Integration Testing: End-to-end workflows (application → lease → payment)
UI Testing: Cross-device manual testing (desktop & mobile)
Real-time Testing: Socket.IO chat functionality
Background Jobs: Verified via Bull Board monitoring
Security Testing: JWT auth, role-based access, input validation

Conclusions and Future Work

Conclusion

MavHousing transforms campus living by bringing applications, leasing, maintenance, and payments into a single, easy-to-use platform. Designed for both students and staff, it combines a highly scalable architecture with smart AI features to automate administrative tasks, ultimately delivering a faster, more reliable housing experience for everyone.

Future work

Future enhancements will focus on integrating secure Stripe/ACH payments, launching native iOS and Android apps, and introducing AI-powered roommate matching. We also plan to implement automated lease renewals and ensure full WCAG accessibility compliance across the platform.

References

[1] Next.js Documentation. *React Framework for Production*. https://nextjs.org/docs
[2] NestJS Documentation. *Progressive Node.js Framework*. https://docs.nestjs.com
[3] Prisma Documentation. *Next-generation ORM for Node.js & TypeScript*. https://www.prisma.io/docs
[4] Socket.IO Documentation. *Real-time WebSocket Library*. https://socket.io/docs
[5] Redis Documentation. *In-memory Data Store*. https://redis.io/docs
[6] BullMQ Documentation. *Redis-based Queue System*. https://docs.bullmq.io
[7] Cloudflare R2 Documentation. *S3-compatible Object Storage*. https://developers.cloudflare.com/r2
[8] Swagger OpenAPI Specification. *API Documentation Standard*. https://swagger.io/specification/
[9] Google AI Gemini API Documentation. *Generative AI Models*. https://ai.google.dev